Exchange Online Mailbox Automation with Logic Apps Managed Identity — Replacing the O365 Connector with Microsoft Graph

Implementation Solution Guide

Version 1.0.0 · 2026-07-17

Contents

Executive Summary — Exchange Online Mailbox Automation with Logic Apps Managed Identity (Microsoft Azure) 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 4

Outcome . . . 5

1. Solution Overview — Exchange Online Mailbox Automation with Logic Apps Managed Identity (Replacing the O365 Connector with Microsoft Graph) 5

The problem this solves (root cause, stated plainly) . . . 5

The supported fix — drive the mailbox through Microsoft Graph . . . 5

Webhook vs polling — how the workflow learns there is new mail . . . 6

How access is scoped to one mailbox (concise — full walkthrough is a separate product) . . . 7

Benefits . . . 7

Limitations (be honest with buyers) . . . 7

The standard deployment model . . . 8

2. Requirements Guide — Exchange Online Mailbox Automation with Logic Apps Managed Identity (Graph) 8

2.1 An Azure subscription and a Logic App . . . 8

2.2 A managed identity on the Logic App (system- or user-assigned) . . . 8

2.3 Exchange Online and the target mailbox . . . 9

2.4 Ability to grant an application Graph permission (admin consent) . . . 9

2.5 A mailbox-scoping control (one-mailbox restriction) . . . 9

2.6 Microsoft.Graph PowerShell for the scripts . . . 9

2.7 Network / endpoint reachability . . . 10

2.8 Change control, pilot scope, and rollback . . . 10

Checklist . . . 10

Global Requirements — Azure Solutions 10

G1. A Microsoft 365 / Entra tenant . . . 11

G2. Administrative roles (least privilege) . . . 11

G3. Licensing baseline . . . 11

G4. Network connectivity to Microsoft 365 / Azure endpoints . . . 11

G5. Change control & maintenance window . . . 12

G6. Backup / rollback posture . . . 12

3. Implementation Guide — Exchange Online Mailbox Automation with Logic Apps Managed Identity (Graph) 12

Naming (set once, reuse verbatim) . . . 13

Step 1 — Enable the managed identity on the Logic App . . . 13
Step 2 — Grant the application Graph permission (app role) to the identity, with admin consent . . . 13
Step 3 — Scope the mailbox access to exactly one mailbox (concise; deep walkthrough deferred) 14
Step 4 — Build the Graph-based read in the Logic App (managed identity auth) . . . 14
Step 5 — Choose the trigger pattern and go deep with the matching sibling . . . 15
Step 6 — Pilot and validate . . . 15
5. Testing Guide — Exchange Online Mailbox Automation with Logic Apps Managed Identity (Graph) 16
Test 1 — The managed identity exists and authenticates . . . 16
Test 2 — Least privilege: Mail.Read and nothing broader . . . 16
Test 3 — The identity reads the IN-scope mailbox . . . 16
Test 4 — An OUT-of-scope mailbox is DENIED (the security boundary) . . . 16
Test 5 — The workflow processes a test message end to end . . . 17
Test 6 — No stored credential remains . . . 17
Test 7 — Evidence report . . . 17
Pass/fail gate . . . 17
6. Troubleshooting Guide — Exchange Online Mailbox Automation with Logic Apps Managed Identity (Graph) 17
A. The trigger won't accept the managed identity (the root cause) . . . 18
B. Admin consent / permission not granted . . . 18
C. 403 reading the mailbox (scope / permission) . . . 18
D. Token / audience errors . . . 19
E. Managed identity not enabled / not found . . . 19
F. Scripts (Microsoft.Graph) issues . . . 20
G. When to escalate . . . 20
7. Best Practices — Exchange Online Mailbox Automation with Logic Apps Managed Identity (Graph) 20
8. Deliverables Summary — Exchange Online Mailbox Automation with Logic Apps Managed Identity (Graph) 21
Documentation . . . 21
PowerShell scripts (Microsoft.Graph) . . . 22
What the buyer receives . . . 22
Related products (the cluster — go deep where you need it) . . . 23
Outcome . . . 23
Appendix A — PowerShell Scripts 24

Executive Summary — Exchange Online Mailbox Automation with Logic Apps Managed Identity (Microsoft Azure)

White-label · Azure Solutions

A team builds an Azure **Logic Apps** workflow to act on an Exchange Online mailbox and tries to authenticate it with the Logic App's **managed identity** using the **Office 365 Outlook** connector's *"When a new email arrives"* trigger — and it simply does not work. This is not a misconfiguration you can fix in the trigger's settings.

Root cause: the Office 365 Outlook connector authenticates as a **user / service account** through a stored OAuth connection. It has no way to authenticate as a managed identity, so the trigger cannot bind to one. The usual "workaround" — pointing the connection at a licensed service-account mailbox — reintroduces exactly the stored-credential and shared-account risk the managed identity was meant to remove. *(Whether any specific connector operation supports managed identity is a product detail that changes — verify against current Microsoft documentation.)*

This solution delivers the **supported fix**: stop using that connector and drive the mailbox through **Microsoft Graph**, with the Logic App's managed identity holding a **least-privilege application permission** (`Mail.Read`) that is **scoped to exactly one mailbox** — not the whole tenant. This is a complete, **deploy-it-today** package written for an administrator with no prior Graph or managed-identity experience.

What you get

- A plain-language **Solution Overview** that states the root cause, presents the Graph design, and gives a **webhook-vs-polling decision table** — pointing to the sibling products that go deep on each pattern, and to the sibling that goes deep on mailbox scoping.
- A **Requirements Guide** with verify / implement-if-missing for every prerequisite.
- An **Implementation Guide** of precise, ordered steps: enable the identity, grant the `Mail.Read` app role with admin consent, **scope the access to one mailbox**, and build the Graph read with managed-identity authentication.
- Five **PowerShell scripts** (Microsoft.Graph) for discovery, least-privilege validation, an application-permission/scope report, the in-scope/out-of-scope **boundary test**, and a `-WhatIf`-defaulted helper that assigns the app role.
- **Testing** and **Troubleshooting** guides, starting with the exact "the trigger won't accept the managed identity" symptom.

What this delivers

A Logic App that reads its mailbox as its **managed identity** — **no service account, no stored OAuth connection** — with a single application permission scoped to a single mailbox and an explicit out-of-scope test that must fail before go-live. The same authenticated, scoped identity then underpins either an event-driven (change-notification) or a polling trigger pattern.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — with a least-privilege, one-mailbox scope as the non-negotiable gate.

Outcome

Mailbox automation that authenticates the modern, supported way — credential-free managed identity over Microsoft Graph, provably limited to one mailbox — instead of a connector that quietly falls back to a stored service-account credential. Deployable by a single administrator and validated before you scale.

1. Solution Overview — Exchange Online Mailbox Automation with Logic Apps Managed Identity (Replacing the O365 Connector with Microsoft Graph)

The problem this solves (root cause, stated plainly)

A common pattern is to build an Azure **Logic Apps** workflow that acts on an Exchange Online mailbox — read new mail, process attachments, trigger downstream work — and to try to authenticate it with the Logic App's **managed identity** by using the **Office 365 Outlook** connector (the *"When a new email arrives"* / *Mail Receive* trigger). **It does not work, and it is not a bug you can configure around.**

Root cause: the **Office 365 Outlook connector authenticates as a user / service account through an interactive OAuth connection** — it is bound to a signed-in mailbox principal, stored as an API connection resource. It has **no mechanism to authenticate as the Logic App's managed identity**. The trigger therefore cannot bind to a system-assigned (or user-assigned) managed identity at all; the connector simply is not a managed-identity-aware connector. *(Whether any specific Office 365 Outlook connector operation or trigger supports managed identity is a product detail that changes over time — **verify against current Microsoft documentation** before relying on it. As of this writing the trigger authenticates via a stored OAuth connection, not a managed identity.)*

The result is a design that forces you to store a **service-account credential** (a licensed mailbox whose OAuth token the connection holds) — the exact stored-secret and shared-account risk that using a managed identity was supposed to eliminate.

The supported fix — drive the mailbox through Microsoft Graph

Stop using the Office 365 Outlook connector for this. Instead, have the Logic App call **Microsoft Graph** directly, authenticating with its **managed identity**, where the identity holds an **application** Graph permission (an **app role**, e.g. `Mail.Read`) and the mailbox access is **scoped so the identity can read exactly one mailbox — not every mailbox in the tenant**.

In this model:

- The Logic App has a **managed identity** — either **system-assigned** (tied to the Logic App's lifecycle) or **user-assigned** (a standalone identity you can share across resources). Either works; this solution uses **system-assigned** by default and notes where user-assigned differs.
- That identity's **service principal** is granted an **application** Microsoft Graph permission — an *app role* such as `Mail.Read` — with **admin consent**. Application permissions are **not** delegated: there is

no signed-in user, and by default an application `Mail.Read` grant can read **every** mailbox in the tenant. That tenant-wide reach is the risk, which is why scoping is mandatory.

• The mailbox access is **scoped to one mailbox** so the identity can read only `automation-inbox@contoso.com`, not the whole tenant. This is the security spine of the design (summarized below; the deep walkthrough is a separate product — see *Mailbox scoping*).

• The Logic App calls the Graph mail endpoints (e.g. `GET /users/{id}/messages`) using its managed identity as the authentication type. **No service account, no stored OAuth connection, no shared password.**

*(Exact Graph permission names and scopes, and the precise Graph endpoints for reading mail, are Microsoft product details — the concepts here are stable, but **verify the exact permission name and endpoint against current Microsoft Graph documentation** before you grant or call them.)*

Webhook vs polling — how the workflow learns there is new mail

Graph gives you two ways to drive a mailbox workflow. This flagship builds the **authentication and scoping foundation** that **both** patterns require; the two patterns themselves are covered in depth by dedicated sibling products.

	Event-driven (webhook / change notifications)	Polling
How it triggers	Graph **change notifications** push to your workflow when mail arrives	The workflow **wakes on a timer** and queries Graph for new mail
Latency	Near-real-time	Bounded by the polling interval
Moving parts	A **subscription** you must **create and renew** before it expires; a validated public HTTPS notification endpoint	A recurrence trigger and a "what have I already seen" watermark
Best when	You need low latency and can operate a renewing subscription + public endpoint	You want the simplest possible design and can tolerate interval latency
Watch-outs (verify)	Max subscription lifetime, renewal cadence, and which mailbox resources support notifications **change — verify against current Microsoft documentation**	Duplicate/last-processed handling; throttling under tight intervals

• **For the event-driven design, go deep with the sibling product: *"Event-Driven Mail Processing with Microsoft Graph Change Notifications (Logic Apps + Managed Identity)."*** It covers subscription creation, the validation handshake, the renewal loop, and delta handling — do not reinvent those here.

• **For the polling design, go deep with the sibling product: *"Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account)."*** It covers the recurrence pattern, watermarking already-processed messages, and throttling-safe intervals.

This product deliberately stops at "the identity can authenticate and read exactly one mailbox." Once that foundation is proven, choose your trigger pattern and follow the matching sibling.

How access is scoped to one mailbox (concise — full walkthrough is a separate product)

An application `Mail.Read` grant is **tenant-wide by default** — the identity could read every mailbox. To reduce it to a single mailbox, Microsoft provides mailbox-scoping controls for application permissions. Conceptually there are two approaches:

- **Application Access Policy** (the long-standing Exchange Online control) — you create a **mail-enabled security group** containing only the target mailbox and bind a policy that restricts the app (the managed identity's service principal, by its app/client ID) to members of that group.
- **Graph RBAC for Exchange Online** (the newer, role-based scoping model) — role assignments scope an app's mailbox access to a defined set of mailboxes.

Which control to use, and its exact syntax, is version- and tenant-dependent — verify against current Microsoft documentation. This flagship includes only the concise "make it one mailbox" step needed to finish the build (Implementation Step 3). **For the full, click-by-click scoping walkthrough — Application Access Policy vs Graph RBAC, when to use each, the exact cmdlets, and how to prove the boundary — use the dedicated sibling product: *"Scoping Application-Permission Mailbox Access — Graph RBAC & Application Access Policies for Exchange Online."*** Do not treat the summary here as the complete scoping design.

Benefits

- **No stored credentials.** The managed identity replaces the service account and its OAuth connection — nothing to rotate, leak, or share.
- **Least privilege, provable.** One application permission, scoped to one mailbox, with an explicit out-of-scope test that must fail.
- **Supported and durable.** Microsoft Graph is the strategic API for mail; the Office 365 Outlook connector's user-auth model is the thing being modernized away.
- **Pattern-ready.** The same authenticated, scoped identity underpins either a webhook or a polling workflow (the two siblings).

Limitations (be honest with buyers)

- **You give up the connector's convenience.** The Office 365 Outlook connector's ready-made triggers/actions are replaced by explicit Graph calls; you build the request, handle paging, and handle the trigger pattern yourself (or via a sibling product's pattern).
- **Application permission is powerful.** `Mail.Read` as an application permission is tenant-wide **until you scope it** — scoping is not optional, it is the design.
- **Consumption vs Standard Logic Apps differ.** How the managed identity is enabled and referenced, and some workflow features, differ between **Logic Apps Consumption** and **Logic Apps Standard** — this guide uses **Consumption** by default and flags where Standard differs; **verify the specifics against current Microsoft documentation.**

• **National clouds differ.** GCC High / DoD and other national clouds use **different Graph and login endpoints** and may differ in feature availability — **verify the sovereign-cloud endpoints and parity against current Microsoft documentation** before deploying there.

• This solution provisions **read** access (`Mail.Read`) as the worked example. Send/modify scenarios need different (higher) permissions and their own scoping and approval.

The standard deployment model

> ***Assess*** *the current (broken) connector-based workflow and the target mailbox →* ***Configure*** *the Logic App managed identity, the application Graph permission (app role), and the one-mailbox scope →* ***Pilot*** *a Graph-based read against the single mailbox →* ***Validate*** *that in-scope reads succeed and out-of-scope reads are denied and a test message flows end to end →* ***Expand*** *to the chosen trigger pattern (webhook or polling sibling) and to additional mailboxes/workflows as needed.*

Repeatable pattern: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with a **least-privilege, one-mailbox scope** as the non-negotiable gate before any workflow logic is trusted.

2. Requirements Guide — Exchange Online Mailbox Automation with Logic Apps Managed Identity (Graph)

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-azure-solutions.md`. Objects are named consistently and reused verbatim (see the naming table in `03-implementation.md`).

> *The security spine of this solution is a* ***least-privilege application Graph permission scoped to ONE mailbox****. Every requirement below serves that goal.*

2.1 An Azure subscription and a Logic App

Description: An Azure subscription and an Azure **Logic App** to host the workflow — named `la-mailbox-automation-pilot`. This guide uses **Logic Apps Consumption** by default. **Why:** The Logic App is the workload that will hold the managed identity and call Microsoft Graph; the managed identity only exists once the Logic App exists. **Verify:** `portal.azure.com` → **Logic apps** → confirm (or plan) `la-mailbox-automation-pilot`; run `LAMIMAIL-Discovery.ps1`. **Implement-if-missing:** Create the Logic App in the pilot resource group `rg-mailbox-automation-pilot`. **Verify with Microsoft: Consumption vs Standard Logic Apps differ** in how the managed identity is enabled and referenced and in some workflow features — if you must use **Standard**, **verify the managed-identity and Graph-call specifics against current Microsoft documentation**; this guide's steps assume Consumption.

2.2 A managed identity on the Logic App (system- or user-assigned)

Description: A **managed identity** for `la-mailbox-automation-pilot` — **system-assigned** by default (this guide), or a **user-assigned** identity `id-mailbox-automation-pilot` if you need to share it across resources. **Why:** The managed identity is what authenticates to Graph with no stored credential — it replaces the Office 365 Outlook connector's service-account OAuth connection. **Verify:**

`portal.azure.com` → the Logic App → **Identity** → **System assigned** = **On** (or a user-assigned identity is attached); run `LAMIMAIL-Discovery.ps1`. **Implement-if-missing:** Turn **System assigned** identity **On** (Implementation Step 1), or create and attach `id-mailbox-automation-pilot`.

2.3 Exchange Online and the target mailbox

Description: Exchange Online is active, and the **single target mailbox** the workflow will read — `automation-inbox@contoso.com` — exists. **Why:** This mailbox is the one (and only) mailbox the identity will be scoped to read; the whole design targets exactly this object. **Verify:** `admin.microsoft.com` → Exchange admin center → **Recipients** → **Mailboxes** → confirm `automation-inbox@contoso.com`; run `LAMIMAIL-Discovery.ps1`. **Implement-if-missing:** Create or designate the mailbox; record its address and object ID (used by the scope and the scripts).

2.4 Ability to grant an application Graph permission (admin consent)

Description: An account able to grant **application** Microsoft Graph permissions (app roles) to a service principal and **provide admin consent** — typically **Global Administrator** or **Privileged Role Administrator**. **Why:** Assigning the app role (e.g. `Mail.Read`) to the managed identity's service principal and consenting to it requires a privileged role; without it the identity cannot read any mail. **Verify:** Entra admin center → **Roles & admins** → confirm the operating account holds the role. **Implement-if-missing:** Have a Global Administrator perform (or delegate) the app-role assignment and consent in Implementation Step 2. **Verify with Microsoft:** the **exact application permission name/scope** to grant for reading mail (e.g. `Mail.Read`) is a Graph product detail — **verify the current permission name and its exact effect against Microsoft Graph documentation** before consenting.

2.5 A mailbox-scoping control (one-mailbox restriction)

Description: A decision on the **mailbox-scoping control** that restricts the application permission to only `automation-inbox@contoso.com` — an **Application Access Policy** (with a mail-enabled security group `sg-MailboxAutomation-Scope-Pilot`) or **Graph RBAC for Exchange Online**. **Why:** An application `Mail.Read` grant is **tenant-wide by default**; scoping to one mailbox is the security requirement of this solution, not an optional hardening step. **Verify:** Confirm the scope binding exists and lists only the target mailbox; run `LAMIMAIL-Test-MailboxAccess.ps1` (in-scope read succeeds, out-of-scope read denied). **Implement-if-missing:** Apply the concise scope in Implementation Step 3; for the full walkthrough use the sibling product *"Scoping Application-Permission Mailbox Access — Graph RBAC & Application Access Policies for Exchange Online."* **Verify with Microsoft: whether to use Application Access Policy or the newer Graph RBAC-for-Exchange model, and their exact cmdlets/syntax, is version- and tenant-dependent — verify against current Microsoft documentation.**

2.6 Microsoft.Graph PowerShell for the scripts

Description: The **Microsoft.Graph** PowerShell SDK installed on the operator workstation (for the LAMIMAIL discovery/validation/reporting/test scripts). **Why:** The included scripts authenticate with `Connect-MgGraph` and read identity, app-role, and mailbox-scope state through Graph. **Verify:** `Get-Module Microsoft.Graph -ListAvailable` returns a version; the scripts print a clear message if it is missing. **Implement-if-missing:** `Install-Module Microsoft.Graph -Scope CurrentUser`.

2.7 Network / endpoint reachability

Description: The Logic App runtime and the operator workstation can reach the required **Microsoft Graph** and **login** endpoints. **Why:** The workflow authenticates the managed identity and calls Graph over the network; blocked egress breaks both auth and the mail read. **Verify:** Confirm reachability to the commercial endpoints (`graph.microsoft.com`, `login.microsoftonline.com`); run `LAMIMAIL-Validate.ps1`. **Implement-if-missing:** Open firewall/proxy allowances for the correct endpoint set. **Verify with Microsoft: national clouds use different endpoints** (e.g. `graph.microsoft.us`, `login.microsoftonline.us` and DoD variants) and may differ in feature availability — **verify the sovereign-cloud Graph/login endpoints and parity against current Microsoft documentation** before deploying to GCC High / DoD.

2.8 Change control, pilot scope, and rollback

Description: An approved change window, the named pilot scope (one Logic App, one mailbox), and a documented rollback before granting the app role or applying the scope. **Why:** Granting an application permission and binding a mailbox scope are tenant-affecting changes; the pilot scope and rollback are the safety net. **Verify:** An approved change record, the pilot scope, and a written back-out exist (per `global-requirements-azure-solutions.md` G5/G6). **Implement-if-missing:** Capture current state (`LAMIMAIL-Report.ps1`), define the pilot scope, and write the rollback (remove the app-role assignment; remove the scope binding).

Checklist

- Azure subscription + Logic App `la-mailbox-automation-pilot` (Consumption; Standard = verify) — `LAMIMAIL-Discovery.ps1`
- Managed identity on the Logic App (system-assigned default, or user-assigned `id-mailbox-automation-pilot`)
- Exchange Online + target mailbox `automation-inbox@contoso.com`
- Ability to grant an application Graph permission + admin consent (Global Admin)
- Mailbox-scoping control chosen (App Access Policy or Graph RBAC — verify which)
- Microsoft.Graph PowerShell installed
- Graph / login endpoint reachability (national-cloud endpoints = verify)
- Change window, pilot scope, and rollback documented

Global Requirements — Azure Solutions

These requirements are common to **every** Azure Solutions solution in the standard program model. They are the baseline platform prerequisites that any Azure/Microsoft 365 engagement assumes. Each is structured **Description / Why / Verify / Implement-if-missing**. Solution-specific requirements (see `02-requirements.md`) are layered on top of these.

Naming convention: every created object is named `<Purpose> – <Scope> – <Stage>` and reused verbatim throughout the engagement.

G1. A Microsoft 365 / Entra tenant

• **Description:** An active Microsoft 365 / Microsoft Entra tenant for the cloud(s) in scope (commercial and/or government).

• **Why:** All identity, licensing, and service configuration is anchored to a tenant; nothing can be configured without one.

• **Verify:** Sign in to the relevant admin center (`https://admin.microsoft.com`) and confirm the tenant resolves and is active under **Settings** → **Org settings**.

• **Implement-if-missing:** Provision the tenant through the appropriate channel (commercial: standard; government: via an authorized AOS-G reseller).

G2. Administrative roles (least privilege)

• **Description:** Appropriately scoped admin roles for the work — Global Administrator for tenant-wide changes, and service-specific roles (Exchange, SharePoint, Teams, Azure Owner/Contributor) for day-to-day.

• **Why:** Every configuration and migration action requires a role with sufficient privilege; least-privilege scoping limits blast radius and satisfies audit requirements.

• **Verify:** In Entra → **Roles & admins**, confirm the operating accounts hold the required roles. In Azure → **Subscriptions** → **Access control (IAM)**, confirm RBAC assignments.

• **Implement-if-missing:** Assign the minimum roles needed; prefer eligible (PIM) activation over standing access where available.

G3. Licensing baseline

• **Description:** Sufficient Microsoft 365 and/or Azure licensing/credit for all in-scope users and workloads.

• **Why:** Features and capacity (mailboxes, SharePoint storage, AVD, VMs) are gated by license SKUs and subscription quota.

• **Verify: Billing** → **Licenses** for M365; **Subscriptions** → **Usage + quotas** for Azure. Compare assigned/available against the in-scope object count.

• **Implement-if-missing:** Procure the correct SKUs and request quota increases sized to the workload (add headroom for shared/resource objects).

G4. Network connectivity to Microsoft 365 / Azure endpoints

• **Description:** Reliable network egress to the required Microsoft 365 and Azure service endpoints (and the sovereign `.us` endpoints for government clouds), with sufficient bandwidth.

• **Why:** All cloud administration and any data movement depends on connectivity to the correct endpoint set; government clouds use distinct endpoints from commercial.

• **Verify:** Confirm reachability to the relevant endpoints (commercial `*.microsoft.com` / government `*.microsoft.us`, `*.azure.us`, `*.office365.us`). Measure sustained bandwidth against the data volume.

• **Implement-if-missing:** Open firewall/proxy allowances for the correct endpoint set; scale bandwidth or add ExpressRoute (matching the cloud — Gov ExpressRoute for Gov regions).

G5. Change control & maintenance window

• **Description:** An approved change-management process with scheduled maintenance windows and a stakeholder communications plan.

• **Why:** Cloud changes can be outward-facing and disruptive; controlled windows and comms reduce business impact and satisfy governance/compliance.

• **Verify:** Confirm an approved change record and a scheduled window for each disruptive action.

• **Implement-if-missing:** Stand up a lightweight change/approval process; schedule windows; prepare user comms templates.

G6. Backup / rollback posture

• **Description:** A defined backup and rollback position for any change — config export, data backup, and a documented back-out procedure before any disruptive action.

• **Why:** Cloud changes are not always reversible by default; a tested rollback is the safety net for cutovers and bulk operations.

• **Verify:** Confirm pre-change config/state is captured and a written rollback exists for each disruptive change.

• **Implement-if-missing:** Export current configuration/state; document and (where feasible) rehearse the rollback before executing the change.

3. Implementation Guide — Exchange Online Mailbox Automation with Logic Apps Managed Identity (Graph)

Follow these steps **in order**. Work in the **Azure portal** (`https://portal.azure.com`), the **Entra admin center** (`https://entra.microsoft.com`), and PowerShell (Microsoft.Graph / Exchange Online). Objects are named once below and reused **verbatim** in every later step, script, and test.

> ***Model:*** *Enable the identity → grant the least-privilege application permission (app role) with admin consent →* ***scope the mailbox to exactly one*** *→ build the Graph read → pilot. Do* ***not*** *trust any workflow logic until the out-of-scope read is proven to fail (Testing).* ***Portal navigation paths and Graph permission names below are current-at-writing — treat them as*** `verify-with-Microsoft`***. Confirm each blade name and each permission name against current Microsoft documentation before you click/grant.***

Naming (set once, reuse verbatim)

Object	Name
Resource group	`rg-mailbox-automation-pilot`
Logic App (Consumption)	`la-mailbox-automation-pilot`
Managed identity (default)	**system-assigned** identity of `la-mailbox-automation-pilot`
Managed identity (alternative)	user-assigned `id-mailbox-automation-pilot`
Target mailbox	`automation-inbox@contoso.com`
Application permission (app role)	`Mail.Read` (Microsoft Graph, **application**) — *verify name*
Mail-enabled security group (App Access Policy scope)	`sg-MailboxAutomation-Scope-Pilot`

Step 1 — Enable the managed identity on the Logic App

1. `portal.azure.com` → **Logic apps** → open `la-mailbox-automation-pilot` (create it in `rg-mailbox-automation-pilot` first if it does not exist).

2. In the left menu → **Settings** → **Identity**. *(Verify the exact blade name against current portal navigation.)*

3. On the **System assigned** tab, set **Status** = **On** → **Save** → **Yes** to confirm.

4. Azure creates a **service principal** for the identity and shows its **Object (principal) ID**. **Record this Object ID** — the app-role grant (Step 2) and the scripts use it.

• **User-assigned alternative:** instead create `id-mailbox-automation-pilot` (**Managed Identities** → **Create**), then on the Logic App's **Identity** → **User assigned** tab **Add** it. Record *its* Object ID. **Verify with Microsoft** how a Consumption vs Standard Logic App references a user-assigned identity in a workflow action.

1. Run `LAMIMAIL-Discovery.ps1` to confirm the identity and its service principal are visible.

Step 2 — Grant the application Graph permission (app role) to the identity, with admin consent

The managed identity's service principal must hold the **application** permission `Mail.Read` (Microsoft Graph). Application permissions are granted as **app-role assignments** on the service principal and require **admin consent**. There is no interactive consent screen for a managed identity, so the assignment is made directly (portal Enterprise Applications, or the included helper script).

1. **Option A — helper script (recommended, auditable):** run `LAMIMAIL-Set-AppRole.ps1` to assign the `Mail.Read` app role to the identity's service principal. **It defaults to** `-WhatIf` and only previews the change; re-run with `-Execute` to apply. The script requires a role able to grant application permissions (Global Administrator / Privileged Role Administrator).

2. **Option B — portal:** `entra.microsoft.com` → **Identity** → **Applications** → **Enterprise applications** → set the filter to include **Managed Identities** → open the service principal for `la-mailbox-automation-pilot` (or `id-mailbox-automation-pilot`) → **Permissions** → grant the Microsoft Graph **application** permission `Mail.Read` and **Grant admin consent**. *(Verify the exact blade path and the permission name against current Microsoft documentation — Enterprise Applications navigation and Graph permission names change.)*

3. **Confirm consent:** re-open the service principal's **Permissions** — `Mail.Read` (application) shows as **granted**. Run `LAMIMAIL-Report.ps1` to export the app-role grants on the identity.

> ***Do not stop here.*** *At this point the identity can read* ***every mailbox in the tenant*** *— an application* `Mail.Read` *grant is tenant-wide by default. Step 3 is mandatory.*

Step 3 — Scope the mailbox access to exactly one mailbox (concise; deep walkthrough deferred)

Restrict the application permission so the identity can read **only** `automation-inbox@contoso.com`. Choose one control:

Option A — Application Access Policy (mail-enabled security group):

1. Create a **mail-enabled security group** `sg-MailboxAutomation-Scope-Pilot` and add **only** `automation-inbox@contoso.com` as a member.

2. In Exchange Online PowerShell, create an **application access policy** that restricts the app (identified by the managed identity's **application/client ID**) to members of `sg-MailboxAutomation-Scope-Pilot`, with **AccessRight = RestrictAccess**.

3. **Test the policy** for both an in-scope and an out-of-scope mailbox — it must allow the first and deny the second.

Option B — Graph RBAC for Exchange Online (newer model):

• Assign an Exchange mail-read role to the app, scoped to a management scope that resolves to only `automation-inbox@contoso.com`.

Verify with Microsoft — this is important: which control to use (Application Access Policy vs Graph RBAC-for-Exchange), the exact cmdlet names, parameters, and how long the scope takes to take effect are version- and tenant-dependent. Confirm against current Microsoft documentation before you run them.

> ***For the full click-by-click scoping walkthrough*** *— both controls compared, exact cmdlets, edge cases, and how to prove the boundary —* ***use the dedicated sibling product: *"Scoping Application-Permission Mailbox Access — Graph RBAC & Application Access Policies for Exchange Online."****** *This step is intentionally the summary needed to finish the build; do not treat it as the complete scoping design.*

1. Run `LAMIMAIL-Test-MailboxAccess.ps1` — it must show the **in-scope** mailbox readable and an **out-of-scope** mailbox **denied**. Do not proceed until it does.

Step 4 — Build the Graph-based read in the Logic App (managed identity auth)

Replace the Office 365 Outlook connector with an explicit Graph call authenticated by the managed identity.

1. Open `la-mailbox-automation-pilot` → **Logic app designer**.

2. Add an **HTTP** action to call Microsoft Graph, for example read messages from the target mailbox:

• **Method:** `GET`

• **URI:** `https://graph.microsoft.com/v1.0/users/automation-inbox@contoso.com/messages?$top=10&$select=id,subject,receivedDateTime,from`

• *(Verify the exact Graph endpoint/path and query options against current Microsoft Graph documentation.)*

1. Under the HTTP action's **Authentication**, choose **Managed identity**:

• **Managed identity: System-assigned** (or select `id-mailbox-automation-pilot` for the user-assigned alternative).

• **Audience / Resource:** `https://graph.microsoft.com` — the token audience must be Graph. *(Verify the audience value against current Microsoft documentation; a wrong audience is the classic token/audience failure.)*

1. **Save.** For the very first proof you can use a **Recurrence** trigger (e.g. every hour) so the workflow runs and reads the mailbox — this is only to prove auth + scope end to end. **The production trigger pattern (webhook vs polling) is chosen next.**

2. **Do not build the connector's *"When a new email arrives"* trigger** — it authenticates as a user/service account via a stored OAuth connection and cannot bind to the managed identity (the root cause this solution fixes).

Step 5 — Choose the trigger pattern and go deep with the matching sibling

The foundation (identity + app role + one-mailbox scope + a working Graph read) is now in place. Choose how the workflow learns there is new mail:

• **Event-driven / near-real-time** → build **Graph change-notification subscriptions**. **Follow the sibling product *"Event-Driven Mail Processing with Microsoft Graph Change Notifications (Logic Apps + Managed Identity)"*** for subscription creation, the validation handshake, and the renewal loop. **Verify subscription lifetime and renewal cadence against current Microsoft documentation.**

• **Polling / simplest** → build a **recurrence + query** loop with a processed-message watermark. **Follow the sibling product *"Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account)."***

Step 6 — Pilot and validate

1. Run `LAMIMAIL-Validate.ps1` → confirms the identity exists, holds **only** the least-privilege `Mail.Read` app role, the mailbox is in scope, and the workflow is present. Expect **PASS**.

2. Run `LAMIMAIL-Test-MailboxAccess.ps1` → in-scope read succeeds, out-of-scope read denied.

3. Send a test message to `automation-inbox@contoso.com` and confirm a workflow run reads it (Testing Guide).

4. **Rollback if needed:** remove the app-role assignment (`LAMIMAIL-Set-AppRole.ps1` supports removal preview) and remove the scope binding; the Logic App and mailbox are otherwise unaffected.

Order of operations

`Enable managed identity → grant Mail.Read app role (+ admin consent) → scope to one mailbox (defer deep steps to #4) → build the Graph read with managed-identity auth → choose webhook (#2) or polling (#3) → pilot & validate (in-scope passes, out-of-scope denied)`

5. Testing Guide — Exchange Online Mailbox Automation with Logic Apps Managed Identity (Graph)

Run after Implementation Step 4 (the Graph read is built) and re-run after any change to the app role, the mailbox scope, or the workflow. Objects are referenced by their names from `03-implementation.md` (`la-mailbox-automation-pilot`, `automation-inbox@contoso.com`, `Mail.Read`, `sg-MailboxAutomation-Scope-Pilot`).

> *The two gates that matter most: **(1)** the identity holds **only** the least-privilege permission, and **(2)** the identity can read the **in-scope** mailbox but is **denied** an **out-of-scope** mailbox. A deployment that fails gate 2 has tenant-wide read and must not go live.*

Test 1 — The managed identity exists and authenticates

Do: `portal.azure.com` → `la-mailbox-automation-pilot` → **Identity** → confirm **System assigned** = **On** (or the user-assigned `id-mailbox-automation-pilot` is attached); run `LAMIMAIL-Discovery.ps1`. **Expected:** The identity's service principal is listed with an Object ID and AppId. **PASS/FAIL:** PASS if the identity resolves; FAIL if no service principal is found.

Test 2 — Least privilege: Mail.Read and nothing broader

Do: Run `LAMIMAIL-Validate.ps1`. **Expected:** `Mail.Read` (application) is granted, and **no** broader Graph application permission is present. **PASS/FAIL:** PASS only if both checks pass; FAIL if `Mail.Read` is missing **or** any broader permission is granted.

Test 3 — The identity reads the IN-scope mailbox

Do: Run `LAMIMAIL-Test-MailboxAccess.ps1` with `-InScopeMailbox automation-inbox@contoso.com`. **Expected:** The in-scope read returns messages (READ OK). **PASS/FAIL:** PASS if the in-scope read succeeds; FAIL if it is denied (check the app-role grant/consent and the scope group membership).

Test 4 — An OUT-of-scope mailbox is DENIED (the security boundary)

Do: Run `LAMIMAIL-Test-MailboxAccess.ps1` with `-OutOfScopeMailbox ceo@contoso.com` (any mailbox NOT in `sg-MailboxAutomation-Scope-Pilot`). **Expected:** The out-of-scope read is **refused** (Access Denied / 403 / ErrorAccessDenied). **PASS/FAIL:** PASS **only** if the out-of-scope read is denied. If it **succeeds**, the mailbox scope is not in place — the identity still has tenant-wide read; **FAIL** and apply the scope (Implementation Step 3 / scoping sibling product).

Test 5 — The workflow processes a test message end to end

Do: Send a test email to `automation-inbox@contoso.com`. Trigger `la-mailbox-automation-pilot` (recurrence, or your chosen webhook/polling trigger). Open **Run history**. **Expected:** A run executes, the Graph HTTP action returns **200** with the message in the body, and the workflow's downstream steps act on it. **PASS/FAIL:** PASS if the run succeeds and the test message is read; FAIL on a 401 (auth), 403 (scope/permission), or a token/audience error (see Troubleshooting).

Test 6 — No stored credential remains

Do: Inspect `la-mailbox-automation-pilot` for any **Office 365 Outlook** API connection or service-account credential. **Expected:** There is **no** Office 365 Outlook connector connection and **no** stored mailbox credential — authentication is the managed identity only. **PASS/FAIL:** PASS if no connector connection/credential exists; FAIL if a service-account OAuth connection is still present (remove it — it is the pattern this solution replaces).

Test 7 — Evidence report

Do: Run `LAMIMAIL-Report.ps1 -OutCsv .\LAMIMAIL-permissions.csv`. **Expected:** A table + CSV of the application permission(s) on the identity and the mailbox-scope binding. **PASS/FAIL:** PASS if the report shows `Mail.Read` as the only application permission and the scope binding lists only the target mailbox.

Pass/fail gate

Gate	Requirement
Least-privilege pass	Tests 1–2 pass — the identity exists and holds only `Mail.Read`
Boundary pass (must hold before go-live)	Tests 3–4 pass — in-scope readable, **out-of-scope denied**
End-to-end pass	Test 5 passes — a test message is read by the workflow with managed-identity auth
Hygiene pass	Tests 6–7 pass — no stored credential remains; evidence report is clean

6. Troubleshooting Guide — Exchange Online Mailbox Automation with Logic Apps Managed Identity (Graph)

Use `LAMIMAIL-Discovery.ps1` / `LAMIMAIL-Validate.ps1` for identity + permission state, `LAMIMAIL-Test-MailboxAccess.ps1` for the mailbox boundary, and `LAMIMAIL-Report.ps1` for the audit record. `LAMIMAIL-Set-AppRole.ps1` (defaults to -WhatIf) grants/removes the app role. Objects are referenced by their names from `03-implementation.md`.

A. The trigger won't accept the managed identity (the root cause)

Symptom	Cause	Fix
The **Office 365 Outlook** *"When a new email arrives"* trigger has no option to use the managed identity; it only offers a sign-in / OAuth connection	**This is expected — it is the root cause.** The Office 365 Outlook connector authenticates as a **user/service account** via a stored OAuth connection; it is not a managed-identity-aware connector and cannot bind to an MI	**Stop using that connector.** Read the mailbox via a Microsoft Graph **HTTP** action with **Managed identity** authentication (Implementation Step 4). Choose the trigger pattern from a sibling product (change notifications for webhook, polling for polling)
A colleague "made it work" with the connector	They pointed the connection at a **licensed service-account mailbox** — reintroducing the stored credential this solution removes	Replace the connection with the Graph + managed-identity design; delete the service-account connection (Testing Test 6)

> ***Verify with Microsoft:*** *whether any specific Office 365 Outlook connector operation supports managed identity can change — confirm against current Microsoft documentation, but the trigger's user/service-account auth model is the reason the MI approach fails.*

B. Admin consent / permission not granted

Symptom	Cause	Fix
Graph call returns **403** and the identity shows no application permission	The `Mail.Read` **app role** was never assigned to the identity's service principal, or admin consent was not given	Grant it: `LAMIMAIL-Set-AppRole.ps1 -IdentityDisplayName la-mailbox-automation-pilot -Execute` (a Global Admin / Privileged Role Admin must run it), or grant via Enterprise Applications → the identity's SP → Permissions. Re-check with `LAMIMAIL-Validate.ps1`
The permission name isn't found	Wrong or changed permission name	**Verify the exact application permission name against current Microsoft Graph documentation**; pass it via `-Permission`
Consent seems granted but reads still fail immediately	Directory propagation delay	Allow time after granting, then re-run `LAMIMAIL-Test-MailboxAccess.ps1`

C. 403 reading the mailbox (scope / permission)

Symptom	Cause	Fix
In-scope read of `automation-inbox@contoso.com` returns **403 ErrorAccessDenied** even though `Mail.Read` is granted	The **Application Access Policy** (or Graph RBAC scope) does not include the target mailbox, or the app's AppId is scoped out	Confirm the mailbox is a member of `sg-MailboxAutomation-Scope-Pilot` and the policy binds the identity's **AppId** to it with **RestrictAccess**; allow the scope time to propagate. See the scoping sibling product
Out-of-scope read **succeeds** (e.g. `ceo@contoso.com` is readable)	**No mailbox scope is in place** — the application `Mail.Read` grant is tenant-wide	This is a security failure. Apply the one-mailbox scope (Implementation Step 3 / scoping sibling) and re-run `LAMIMAIL-Test-MailboxAccess.ps1` until the out-of-scope read is denied

> ***Verify with Microsoft:*** *whether to use Application Access Policy or Graph RBAC-for-Exchange, and the exact cmdlets/propagation time, are version-dependent — confirm against current Microsoft documentation.*

D. Token / audience errors

Symptom	Cause	Fix
Graph HTTP action returns **401** or "invalid audience" / "wrong resource"	The managed-identity authentication's **audience/resource** is not Microsoft Graph	Set the audience to `https://graph.microsoft.com` (or the correct **national-cloud** audience — **verify against current Microsoft documentation**) in the HTTP action's authentication
Works in one cloud, fails in GCC High / DoD	National clouds use **different Graph/login endpoints and audiences**	Use the sovereign endpoints/audiences; verify parity and availability against current Microsoft documentation

E. Managed identity not enabled / not found

Symptom	Cause	Fix
No service principal for `la-mailbox-automation-pilot`; the HTTP action can't select an identity	The identity was never enabled	Logic App → **Identity** → **System assigned** = **On** → Save (Implementation Step 1); or attach `id-mailbox-automation-pilot`. Re-run `LAMIMAIL-Discovery.ps1`
User-assigned identity not selectable in the action	It isn't attached to this Logic App, or the SKU references it differently	Attach it on the **Identity** → **User assigned** tab; **verify Consumption vs Standard referencing against current Microsoft documentation**

F. Scripts (Microsoft.Graph) issues

• `Connect-MgGraph` **scopes.** The scripts request delegated scopes such as `Application.Read.All`, `AppRoleAssignment.ReadWrite.All` (write path only, in `LAMIMAIL-Set-AppRole.ps1`), `User.Read.All`, `Directory.Read.All`. Consent to them when prompted. **Verify the exact scope names against current Microsoft Graph documentation.**

• **Exact display-name matching.** Discovery/Validation/Report/Set-AppRole resolve the identity by **exact** `displayName` (`la-mailbox-automation-pilot`). If your Logic App or user-assigned identity uses a different name, pass the exact name via `-IdentityDisplayName` — a mismatch reports "no service principal found."

• **-WhatIf default.** `LAMIMAIL-Set-AppRole.ps1` changes nothing until `-Execute`. If "nothing happened," you were in the default preview mode — re-run with `-Execute`. Use `-Verify` to report the current grant and `-Remove` to roll back.

• **Module missing.** `Install-Module Microsoft.Graph -Scope CurrentUser` (and `Install-Module ExchangeOnlineManagement` for the scope-binding portion of the report). `LAMIMAIL-Validate.ps1`'s workflow-present check additionally uses `Az.LogicApp` when you pass `-LogicAppName`/`-ResourceGroup`.

• **App-only boundary test.** `LAMIMAIL-Test-MailboxAccess.ps1` connects **app-only** with the app's AppId + a certificate (recommended) or secret, so it exercises the same application-permission + scope the Logic App identity uses. **Verify your app-only connection method against current Microsoft Graph documentation.**

G. When to escalate

If the identity is enabled, `Mail.Read` is granted and consented, the mailbox scope is confirmed and has had time to propagate, the token audience is Graph, and the in-scope read still returns 403 (or the out-of-scope read still succeeds), collect `LAMIMAIL-Report.ps1` output plus the Logic App run history (HTTP action request/response) and open a Microsoft support case.

7. Best Practices — Exchange Online Mailbox Automation with Logic Apps Managed Identity (Graph)

• **Don't fight the connector — replace it.** The Office 365 Outlook connector authenticates as a user/service account via a stored OAuth connection; it cannot bind to a managed identity. Drive the mailbox through Microsoft Graph with the identity instead of trying to force the connector to use the MI.

• **Scope to one mailbox — it is the design, not hardening.** An application `Mail.Read` grant is tenant-wide by default. Always restrict it to a single mailbox (Application Access Policy or Graph RBAC-for-Exchange) and **prove** the boundary with an out-of-scope read that must fail. Never ship on a tenant-wide grant.

• **Grant the least permission that works.** Use `Mail.Read` for read-only workflows; do not reach for `Mail.ReadWrite` or `Mail.Send` unless the workflow genuinely needs them, and scope those the same way. Verify the exact permission name against current Microsoft documentation before granting.

• **Prefer system-assigned identity for a single workload.** It shares the Logic App's lifecycle — deleted with the app, nothing left orphaned. Use a **user-assigned** identity only when several resources must share one identity.

• **Name everything and reuse the name.** `la-mailbox-automation-pilot`, `automation-inbox@contoso.com`, `sg-MailboxAutomation-Scope-Pilot` — reuse verbatim across the workflow, the scope, the scripts, and the tests so the deployment stays auditable and cleanable.

• **Keep the state-changer on -WhatIf.** `LAMIMAIL-Set-AppRole.ps1` defaults to preview; run it read-only first, confirm the change, then `-Execute`. Keep the removal path handy as your rollback.

• **Pick the trigger pattern deliberately.** Webhook (change notifications) for near-real-time; polling for simplicity. Don't hand-roll either — use the matching sibling product (change-notifications for webhook, polling workflows for polling) and verify subscription lifetime/renewal against current Microsoft documentation.

• **Set the token audience to Graph.** In the Logic App HTTP action's managed-identity authentication, the audience/resource must be `https://graph.microsoft.com` (or the correct national-cloud audience). A wrong audience is the classic token error.

• **Mind Consumption vs Standard.** How the identity is enabled and referenced differs between Logic Apps Consumption and Standard; validate on the SKU you will run and verify the specifics against current Microsoft documentation.

• **National clouds are different.** GCC High / DoD use different Graph and login endpoints and can differ in feature parity — verify the sovereign endpoints and availability before deploying there; do not assume the commercial steps transfer unchanged.

• **Pilot, prove, then expand.** One Logic App, one mailbox, both tests green (in-scope pass, out-of-scope deny) before you add mailboxes, workflows, or the production trigger pattern.

8. Deliverables Summary — Exchange Online Mailbox Automation with Logic Apps Managed Identity (Graph)

Documentation

Document	Purpose
Executive Summary	The business case: why the O365-connector-plus-managed-identity approach fails and what the Graph fix delivers
1. Solution Overview	Root cause stated plainly, the supported Graph design, webhook-vs-polling decision table with pointers to the sibling products, and a concise mailbox-scoping section (deep walkthrough deferred to the scoping sibling)

Document	Purpose
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing (subscription + Logic App, managed identity, Exchange Online + mailbox, app-permission consent, Microsoft.Graph PowerShell, network)
Global Requirements (Azure Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: enable the identity, grant the `Mail.Read` app role with admin consent, scope to one mailbox, build the Graph read with managed-identity auth, pilot
5. Testing Guide	Validation with PASS/FAIL gates — least privilege, the in-scope/out-of-scope boundary, and end-to-end message processing
6. Troubleshooting Guide	Symptom → cause → fix, starting with "the trigger won't accept the managed identity"
7. Best Practices	Field-proven recommendations

PowerShell scripts (Microsoft.Graph)

Script	What it does
`LAMIMAIL-Discovery.ps1`	Read-only: reports the Logic App's managed identity, the Graph application permissions (app roles) on it, and whether the target mailbox is reachable
`LAMIMAIL-Validate.ps1`	PASS/FAIL: identity exists, holds **only** the least-privilege `Mail.Read`, mailbox resolves, workflow present
`LAMIMAIL-Report.ps1`	Reports the application-permission grants on the identity + the mailbox-scope binding; CSV export
`LAMIMAIL-Test-MailboxAccess.ps1`	Read-only boundary test: proves the identity reads the IN-scope mailbox and is DENIED an OUT-of-scope mailbox
`LAMIMAIL-Set-AppRole.ps1`	The only state-changer — assigns/verifies/removes the `Mail.Read` app role on the identity; **defaults to -WhatIf**

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the standard model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — with a least-privilege, one-mailbox scope as the non-negotiable gate.

Related products (the cluster — go deep where you need it)

- **Event-driven trigger:** *"Event-Driven Mail Processing with Microsoft Graph Change Notifications (Logic Apps + Managed Identity)."*
- **Polling trigger:** *"Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account)."*
- **Mailbox scoping (deep):** *"Scoping Application-Permission Mailbox Access — Graph RBAC & Application Access Policies for Exchange Online."*

Outcome

A Logic App that reads its target Exchange Online mailbox using its **managed identity** and a **least-privilege application permission scoped to exactly one mailbox** — no service account, no stored OAuth connection — with the out-of-scope boundary proven before go-live, ready for either a webhook or a polling trigger pattern.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

LAMIMAIL-Discovery.ps1

Purpose: Read-only readiness snapshot for the "Exchange Online Mailbox Automation with Logic Apps Managed Identity (Graph)" solution. Reports: the managed identity's service principal (system- or user-assigned) by display name; the Microsoft Graph APPLICATION permissions (app-role assignments) granted to that identity — highlighting anything broader than the least-privilege 'Mail.Read'; whether the target mailbox resolves in the directory (reachability check). Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\LAMIMAIL-Discovery.ps1 `-IdentityDisplayName 'la-mailbox-automation-pilot'` -TargetMailbox 'automation-inbox@contoso.com' REQUIRED GRAPH SCOPES (delegated, for the operator running this script) Application.Read.All, AppRoleAssignment.Read.All, User.Read.All, Directory.Read.All (verify the exact scope names against current Microsoft Graph documentation) EXPECTED RESULTS A printed snapshot: the identity's service principal, each Graph app-role granted to it (with a warning if any grant is broader than Mail.Read), and whether the target mailbox resolves — the readiness picture before validating least privilege and scope. .NOTES Azure Solutions — read-only. The repeatable way. Reports on configuration/identifiers only; makes no changes to identities, permissions, or mailboxes. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$IdentityDisplayName, # e.g. la-mailbox-automation-pilot (or id-mailbox-automation-pilot) [Parameter(Mandatory)] [string]$TargetMailbox, # e.g. automation-inbox@contoso.com [string]$LeastPrivPermission = 'Mail.Read')

How to run: powershell -ExecutionPolicy Bypass -File .\LAMIMAIL-Discovery.ps1 `-IdentityDisplayName 'la-mailbox-automation-pilot'` -TargetMailbox 'automation-inbox@contoso.com' REQUIRED GRAPH SCOPES (delegated, for the operator running this script) Application.Read.All, AppRoleAssignment.Read.All, User.Read.All, Directory.Read.All (verify the exact scope names against current Microsoft Graph documentation) EXPECTED RESULTS A printed snapshot: the identity's service principal, each Graph app-role granted to it (with a warning if any grant is broader than Mail.Read), and whether the target mailbox resolves — the readiness picture before validating least privilege and scope. .NOTES Azure Solutions — read-only. The repeatable way. Reports on configuration/identifiers only; makes no changes to identities, permissions, or mailboxes. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$IdentityDisplayName, # e.g. la-mailbox-automation-pilot (or id-mailbox-automation-pilot) [Parameter(Mandatory)] [string]$TargetMailbox, # e.g. automation-inbox@contoso.com [string]$LeastPrivPermission = 'Mail.Read')

LAMIMAIL-Report.ps1

Purpose: Evidence/audit report for the "Exchange Online Mailbox Automation with Logic Apps Managed Identity (Graph)" solution. Produces two things: a row per Microsoft Graph APPLICATION permission (app role) granted to the identity's service principal (permission name, app-role id, and whether it equals the least-privilege target Mail.Read); the mailbox-scope binding as reported by Exchange Online — the Application Access Policy scoping the app (by AppId) to the scope group, when the ExchangeOnlineManagement module is available (best-effort). (Graph RBAC-for-Exchange scoping: verify + report via that model's cmdlets — see the scoping sibling product.) Makes NO changes. Exports the permission report to CSV. HOW TO RUN powershell -ExecutionPolicy Bypass -File

.\LAMIMAIL-Report.ps1 `-IdentityDisplayName 'la-mailbox-automation-pilot'` -OutCsv .\LAMIMAIL-permissions.csv REQUIRED GRAPH SCOPES (delegated, for the operator) Application.Read.All, AppRoleAssignment.Read.All, Directory.Read.All (verify the exact scope names against current Microsoft Graph documentation) EXPECTED RESULTS A printed table of the identity's Graph application permissions plus (best-effort) the Application Access Policy binding, and a CSV of the permission grants for the audit record. .NOTES Azure Solutions — read-only. The repeatable way. Reporting only; makes no changes. Exchange scope reporting is best-effort and depends on the ExchangeOnlineManagement module and the scoping model in use (App Access Policy vs Graph RBAC). > [CmdletBinding()] param([Parameter(Mandatory)] [string]$IdentityDisplayName, [string]$OutCsv = ".\LAMIMAIL-permissions.csv", [string]$LeastPrivPermission = 'Mail.Read')

How to run: powershell -ExecutionPolicy Bypass -File .\LAMIMAIL-Report.ps1 `-IdentityDisplayName 'la-mailbox-automation-pilot'` -OutCsv .\LAMIMAIL-permissions.csv REQUIRED GRAPH SCOPES (delegated, for the operator) Application.Read.All, AppRoleAssignment.Read.All, Directory.Read.All (verify the exact scope names against current Microsoft Graph documentation) EXPECTED RESULTS A printed table of the identity's Graph application permissions plus (best-effort) the Application Access Policy binding, and a CSV of the permission grants for the audit record. .NOTES Azure Solutions — read-only. The repeatable way. Reporting only; makes no changes. Exchange scope reporting is best-effort and depends on the ExchangeOnlineManagement module and the scoping model in use (App Access Policy vs Graph RBAC). > [CmdletBinding()] param([Parameter(Mandatory)] [string]$IdentityDisplayName, [string]$OutCsv = ".\LAMIMAIL-permissions.csv", [string]$LeastPrivPermission = 'Mail.Read')

LAMIMAIL-Set-AppRole.ps1

Purpose: The one state-changing helper for the "Exchange Online Mailbox Automation with Logic Apps Managed Identity (Graph)" solution. It grants the Microsoft Graph APPLICATION permission 'Mail.Read' (an app-role assignment) to the managed identity's service principal — the grant that lets the identity read mail without a stored credential. It can also -Verify (report only) or -Remove (rollback) the grant.

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\LAMIMAIL-Set-AppRole.ps1 -IdentityDisplayName 'la-mailbox-automation-pilot' Verify only (report the current grant): powershell -ExecutionPolicy Bypass -File .\LAMIMAIL-Set-AppRole.ps1 -IdentityDisplayName 'la-mailbox-automation-pilot' -Verify Apply for real: powershell -ExecutionPolicy Bypass -File .\LAMIMAIL-Set-AppRole.ps1 -IdentityDisplayName 'la-mailbox-automation-pilot' -Execute Rollback (remove the grant), preview then execute: powershell -ExecutionPolicy Bypass -File .\LAMIMAIL-Set-AppRole.ps1 -IdentityDisplayName 'la-mailbox-automation-pilot' -Remove powershell -ExecutionPolicy Bypass -File .\LAMIMAIL-Set-AppRole.ps1 -IdentityDisplayName 'la-mailbox-automation-pilot' -Remove -Execute REQUIRED GRAPH SCOPES (delegated, for the operator running this script) Application.Read.All, AppRoleAssignment.ReadWrite.All, Directory.Read.All (verify the exact scope names against current Microsoft Graph documentation) EXPECTED RESULTS -WhatIf (default): a printed description of the app-role grant/removal — nothing changes. -Verify: reports whether the identity already holds Mail.Read. -Execute: the app-role assignment is created (or removed with -Remove). .NOTES Azure Solutions — DEFAULTS TO -WhatIf. The repeatable way. Grants ONLY the least-privilege Mail.Read application permission; does not scope the mailbox and does not grant anything broader. Follow with the mailbox scope + the boundary test. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$IdentityDisplayName, # e.g. la-mailbox-automation-pilot [string]$Permission = 'Mail.Read', # the least-privilege application permission [switch]$Verify, # report only [switch]$Remove, # rollback: remove the grant [switch]$Execute # omit for -WhatIf (default safe mode))

LAMIMAIL-Test-MailboxAccess.ps1

Purpose: The boundary test for the "Exchange Online Mailbox Automation with Logic Apps Managed Identity (Graph)" solution. Using APPLICATION authentication as the managed identity's app registration (app-only Graph, the same identity the Logic App uses), it attempts to read messages from: the IN-scope mailbox (automation-inbox@contoso.com) — MUST succeed; an OUT-of-scope mailbox (a control mailbox you name) — MUST be denied (403 / ErrorAccessDenied). A correctly scoped deployment PASSES only when the in-scope read works AND the out-of-scope read is denied. Read-only: it only reads (or is refused); it never writes or deletes mail.

How to run: certificate-based app-only auth (recommended) powershell -ExecutionPolicy Bypass -File .\LAMIMAIL-Test-MailboxAccess.ps1 `-TenantId '<tenant-guid>' -AppId '<identity-appId>' -CertThumbprint '<thumbprint>'` -InScopeMailbox 'automation-inbox@contoso.com' ` -OutOfScopeMailbox 'ceo@contoso.com' REQUIRED GRAPH PERMISSION (application, on the app being tested) Mail.Read (application) — scoped to ONE mailbox by Application Access Policy or Graph RBAC. (verify the exact permission name against current Microsoft Graph documentation) EXPECTED RESULTS PASS: in-scope read returns messages; out-of-scope read is refused (Access Denied). Any other combination is a FAIL — in particular, an out-of-scope read that SUCCEEDS means the mailbox scope is not in place (the identity still has tenant-wide read). .NOTES Azure Solutions — read-only boundary test. The repeatable way. Reads (or is denied) mail only; never writes/deletes. A successful out-of-scope read = scope FAIL. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$TenantId, [Parameter(Mandatory)] [string]$AppId, # the managed identity's / app registration's AppId [string]$CertThumbprint, # certificate app-only auth (recommended) [string]$ClientSecret, # OR client-secret app-only auth [Parameter(Mandatory)] [string]$InScopeMailbox, # e.g. automation-inbox@contoso.com (MUST succeed) [Parameter(Mandatory)] [string]$OutOfScopeMailbox # e.g. ceo@contoso.com (MUST be denied))

LAMIMAIL-Validate.ps1

Purpose: Deterministic readiness gate for the "Exchange Online Mailbox Automation with Logic Apps Managed Identity (Graph)" solution. Checks four things and prints a clear PASS/FAIL summary: 1. the managed identity's service principal exists; 2. it holds the least-privilege application permission 'Mail.Read' AND holds NO broader Graph application permission (least privilege is the security spine); 3. the target mailbox resolves (scope is proven separately by LAMIMAIL-Test-MailboxAccess.ps1); 4. the Logic App workflow resource is present (best-effort, via Az if available). Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\LAMIMAIL-Validate.ps1 `-IdentityDisplayName 'la-mailbox-automation-pilot'` -TargetMailbox 'automation-inbox@contoso.com' ` -LogicAppName 'la-mailbox-automation-pilot' -ResourceGroup 'rg-mailbox-automation-pilot' REQUIRED GRAPH SCOPES (delegated, for the operator) Application.Read.All, AppRoleAssignment.Read.All, User.Read.All, Directory.Read.All (verify the exact scope names against current Microsoft Graph documentation) EXPECTED RESULTS A PASS/FAIL line per check and an overall PASS only when the identity exists, is least-privilege (Mail.Read and nothing broader), the mailbox resolves, and the workflow is present. .NOTES Azure Solutions — read-only. The repeatable way. Validation only; makes no changes. Overall FAIL if any required check fails. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$IdentityDisplayName, [Parameter(Mandatory)] [string]$TargetMailbox, [string]$LogicAppName, [string]$ResourceGroup, [string]$LeastPrivPermission = 'Mail.Read')

How to run: powershell -ExecutionPolicy Bypass -File .\LAMIMAIL-Validate.ps1 `-IdentityDisplayName 'la-mailbox-automation-pilot'` -TargetMailbox

'automation-inbox@contoso.com' ` -LogicAppName 'la-mailbox-automation-pilot' -ResourceGroup 'rg-mailbox-automation-pilot' REQUIRED GRAPH SCOPES (delegated, for the operator) Application.Read.All, AppRoleAssignment.Read.All, User.Read.All, Directory.Read.All (verify the exact scope names against current Microsoft Graph documentation) EXPECTED RESULTS A PASS/FAIL line per check and an overall PASS only when the identity exists, is least-privilege (Mail.Read and nothing broader), the mailbox resolves, and the workflow is present. .NOTES Azure Solutions — read-only. The repeatable way. Validation only; makes no changes. Overall FAIL if any required check fails. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$IdentityDisplayName, [Parameter(Mandatory)] [string]$TargetMailbox, [string]$LogicAppName, [string]$ResourceGroup, [string]$LeastPrivPermission = 'Mail.Read')